

Zahlen und Statistisches – Teil 1

static und final

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 12. Januar 2026 }

inno



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

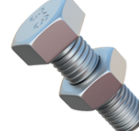
Frei nach Helmuth Karl
Bernhard von Moltke

Zahlen und Statisches

 Plan für diese Woche

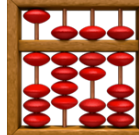
 Statische Methoden

 Statische Variablen

 Konstanten

 Schlüsselwort `final`

 Pair Programming

 Math-Methoden

 Wrapper und Autoboxing

 Zahlenformatierung

 Pair Programming

 Statische Importe

 Spaß

Zahlen und Statisches

Verbindungen herstellen

 Tauschen Sie sich mit einer Nachbar:in aus!

 Was ist der Zusammenhang zwischen statischen Methoden und Instanzvariablen?

 Was ist der Unterschied zwischen statischen und nicht-statischen Variablen?

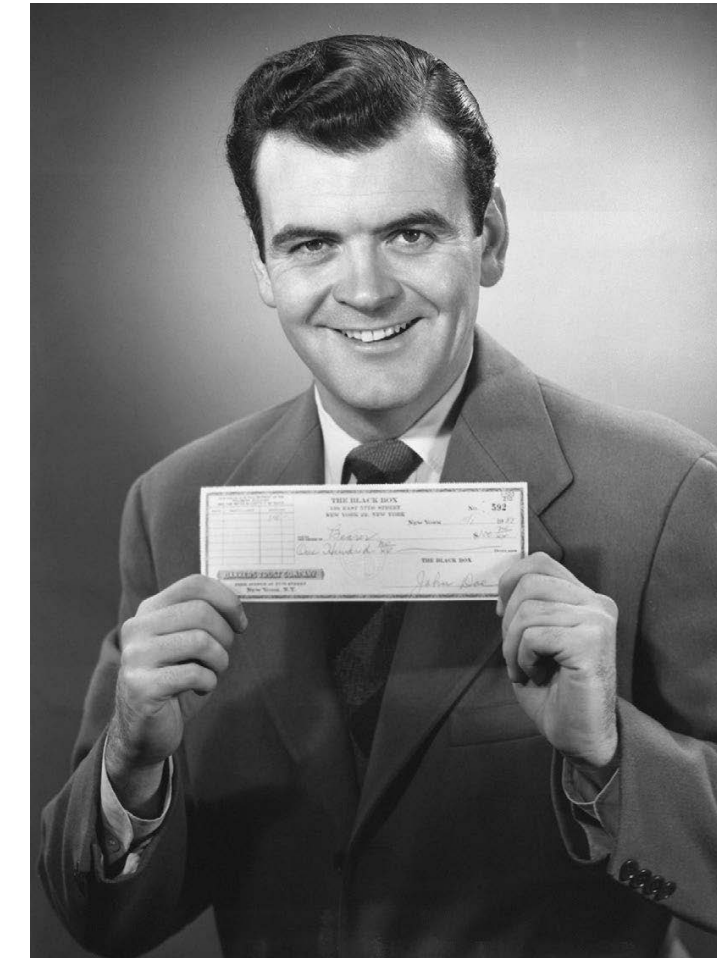
 Wie können Sie in Java Konstanten erzeugen?

 Welche Auswirkungen hat das Schlüsselwort `final` auf Variablen, Methoden und Klassen?

 Welche Fragen haben Sie zu diesem Kapitel?

10 Zahlen und Statisches

Zahlen, bitte!



Rechnen Sie's aus. Zur Arbeit mit Zahlen gehört deutlich mehr als nur primitive Arithmetik. Vielleicht brauchen Sie den absoluten Wert einer Zahl oder wollen eine Zahl runden oder die größere von zwei Zahlen finden. Vielleicht wollen Sie Zahlen mit genau zwei Nachkommastellen ausgeben oder große Zahlen mit Tausenderpunkten versehen, um sie lesbarer zu machen. Und was ist, wenn Sie einen String in eine Zahl umwandeln (parsen) wollen? Oder umgekehrt die Zahl in einen String konvertieren müssen? Eines Tages werden Sie eine Reihe von Zahlen in eine Collection wie eine `ArrayList` stecken wollen, die aber nur Objekte übernimmt. Glücklicherweise sind Java und die Java-API randvoll mit Möglichkeiten und Methoden für die Arbeit mit Zahlen, die Sie direkt und einfach nutzen können. Die meisten von ihnen sind **statisch** (**static**). Also lernen wir zuerst, was es für eine Variable oder Methode bedeutet, statisch zu sein, inklusive der Konstanten in Java, die auch als statische *finale* Variablen bezeichnet werden.



Statische Methoden





„Die Methoden der Klasse Math verwenden keine Werte von Instanzvariablen und müssen daher keine dynamischen Zustände kennen.“

```
long rounded = Math.round(42.2);  
int minimum = Math.min(56, 12);  
int absolute = Math.abs(-343);
```

„Diese Methoden sind also ‚statisch‘. Sie benötigen keine Instanz der Klasse Math, sondern nur die Klasse Math selbst.“

„Aha! Das bedeutet, dass diese Methoden mit ihren Argumenten arbeiten, aber niemals vom Zustand einer Instanzvariablen beeinflusst werden. Das einzige, was zum Beispiel die Ausführung von `min()` beeinflusst, sind die Argumente, die der Methode übergeben werden.“

Gerit – aufgeweckte Student:in im 1. Semester





„Genau, das ist nicht nötig. Es ist sogar unmöglich, eine Instanz der Klasse `Math` zu erzeugen.“

„Mmh, wäre es nicht eine Verschwendung von wertvollem Platz auf dem Heap, extra eine Instanz von `Math` zu erzeugen, nur um die Methode `min()` auszuführen?“

Merle – aufmerksame Student:in im 1. Semester




```
1 public class MathDemo {  
2     public static void main(String[] args) {  
3         Math myMath = new Math();  
4  
5         System.out.println(myMath.round(42.2));  
6         System.out.println(myMath.min(56, 12));  
7         System.out.println(myMath.abs(-343));  
8     }  
9 }
```

10

STDIN

Input for the program (Optional)

Output:

MathDemo.java:3: error: Math() has private access in Math
 Math myMath = new Math();
 ^

1 error

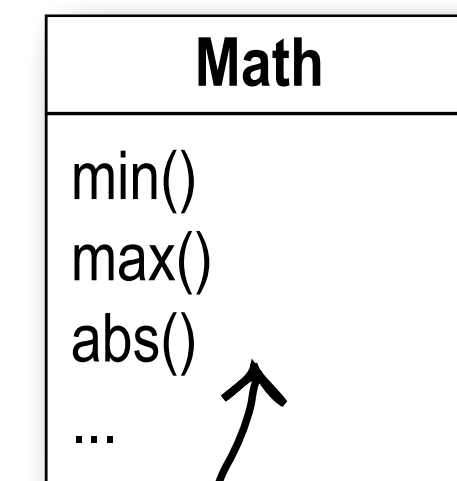
Methoden mit **static**

🔧 Statische Methoden

- Java ist grundsätzlich objektorientiert.
- Es gibt jedoch einige Sonderfälle, wie beispielsweise Hilfsmethoden, für die keine Instanz einer Klasse benötigt wird.
- Eine mit dem Schlüsselwort **static** deklarierte Methode kann **ohne Instanz** einer Klasse ausgeführt werden.

statische Methode

```
public static int min(int a, int b) {  
    // gibt den niedrigeren Wert von  
    // a und b zurück  
}
```



Keine Instanzvariablen. Das Verhalten der Methode wird nicht durch den Zustand von Instanzvariablen beeinflusst.

Math.min(42, 36);

Verwenden Sie den Namen der Klasse anstelle des Namens einer Referenzvariablen.



Nicht-Instanziierbarkeit

🔧 Statische Methoden

- Oft (aber nicht immer) ist es nicht erwünscht, dass eine Klasse mit statischen Methoden instanziiert wird.
- Wenn Sie verhindern wollen, dass Code eine nicht abstrakte Klasse instanziiert, können Sie ihren parameterlosen Konstruktor mit dem Schlüsselwort `private` kennzeichnen.
- Ein als privat markierter Konstruktor verhält sich im Prinzip wie eine private Methode – nur Code innerhalb der Klasse kann den Konstruktor aufrufen.




```
1 public class Cat {
2
3     String name;
4
5     public Cat(String name) {
6         this.name = name;
7     }
8
9     public String getName() {
10        return name;
11    }
12
13    public static void main(String[] args) {
14        // You cannot reference non-static variables from a static context!
15        // The static method does not know which value of the instance variable to use.
16        System.out.println("The name of the cat is: " + name);
17
18        // Similarly, you cannot reference non-static methods from a static context!
19        // Calling getName() just delays the inevitable. The instance variable is used by the getter.
20        System.out.println("The name of the cat is: " + getName());
21    }
22 }
23
```

STDIN

Input for the program (Optional)

Output:

Cat.java:16: error: non-static variable name cannot be referenced from a static context
 System.out.println("The name of the cat is: " + name);
 ^

Cat.java:20: error: non-static method getName() cannot be referenced from a static context
 System.out.println("The name of the cat is: " + getName());
 ^

2 errors

! ? Quiz

🔧 Statische Methoden

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Eine statische Methode sollte über den Klassennamen und nicht über eine Objektreferenzvariable aufgerufen werden:

`Math.random()` vs. `myObj.goStatically()`

Richtig!

Eine statische Methode kann aufgerufen werden, ohne dass eine Instanz der Klasse der Methode auf dem Stack existiert.

... auf dem Heap existiert.

Eine statische Methode eignet sich gut als Hilfsmethode, da sie nicht vom Wert einer bestimmten Instanzvariablen abhängt (und dies auch nie tun wird).

Richtig!

Eine statische Methode ist nicht an eine bestimmte Instanz gebunden, sondern nur an die Klasse. Sie kann also nicht auf die Werte der Instanzvariablen der Klasse zugreifen. Sie würde nicht einmal wissen, welchen konkreten Wert der Instanzvariablen sie verwenden soll.

Richtig!

Eine statische Methode kann nur auf andere statische Methoden zugreifen, da nicht-statische Methoden normalerweise an den Zustand von Instanzvariablen gebunden sind.

Richtig!

Wenn eine Klasse nur statische Methoden enthält und nicht instanziiert werden soll, muss der Konstruktor als privat markiert werden.

... sollte der Konstruktor als privat markiert werden.

 Welcher **eine** Gedanke zu
„ Statische Methoden“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren



Statische Variablen



Geteilte Variablen

🔧 Statische Variablen

- Felder, deren Deklaration den Modifikator **static** enthält, werden **statische** Felder/Variablen oder Klassenvariablen genannt.
- Sie sind an die Klasse und nicht an ein Objekt gebunden.
- Alle Instanzen derselben Klasse teilen sich eine einzige Kopie der statischen Variablen.



```
1 public class StaticFieldDemo {  
2  
3     public static void main(String[] args) {  
4         System.out.println("Creating three counters...");  
5         new Counter();  
6         new Counter();  
7         new Counter();  
8  
9         System.out.println("Creating three *static* counters...");  
10        new StaticCounter();  
11        new StaticCounter();  
12        new StaticCounter();  
13    }  
14 }  
15
```

STDIN

Input for the program (Optional)

Output:

Creating three counters...

Counter: 1

Counter: 1

Counter: 1

Creating three *static* counters...

Static counter: 1

Static counter: 2

Static counter: 3

Initialisierung statischer Variablen

Statische Variablen

- Statische Variablen werden beim Laden einer Klasse initialisiert.
- Eine Klasse wird geladen, weil die JVM entscheidet, dass es an der Zeit ist.
- Typischerweise, weil jemand zum ersten Mal eine neue Instanz der Klasse erzeugt oder eine statische Methode oder Variable der Klasse verwendet.
- Es gibt zwei Garantien für die statische Initialisierung:
 - Statische Variablen einer Klasse werden initialisiert, bevor ein Objekt der Klasse erzeugt wird.
 - Statische Variablen einer Klasse werden initialisiert, bevor eine statische Methode der Klasse ausgeführt wird.


```
1 public class StaticFieldDemo {  
2  
3     public static void main(String[] args) {  
4         System.out.println("Static counter before calling the constructor: " + StaticCounter.counter);  
5  
6         System.out.println("Creating three instances...");  
7         new StaticCounter();  
8         new StaticCounter();  
9         new StaticCounter();  
10    }  
11 }  
12
```

STDIN

Input for the program (Optional)

Output:

```
Static counter before calling the constructor: 0  
Creating three instances...  
Static counter: 1  
Static counter: 2  
Static counter: 3
```


! ? Quiz

🔪 Statische Variablen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Eine statische Variable ist eine Variable, die von allen Mitgliedern einer Klasse gemeinsam genutzt wird. Innerhalb der Klasse ist nur eine Kopie der Variable vorhanden, während bei Instanzvariablen für jedes Objekt eine Kopie angelegt wird.

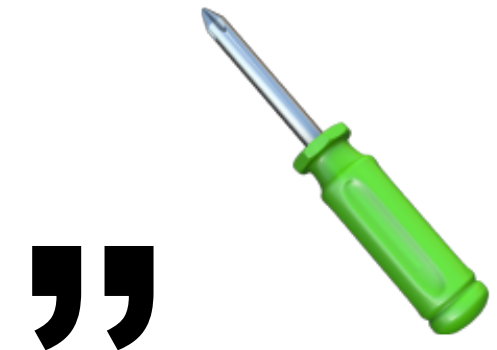
Richtig!

Statische Methoden haben keinen Zugriff auf statische Variablen.

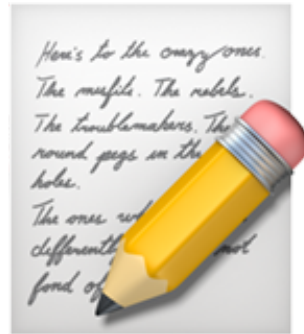
... können auf statische Variablen zugreifen.



Welcher **eine** Gedanke zu



„Statische Variablen“ war
für Sie am wichtigsten?



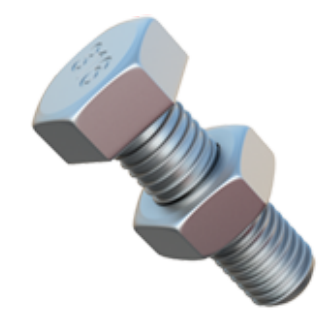
Schreiben Sie ihn auf!



Durchatmen und



Sortieren



Konstanten

Die Variable ist als public markiert, so dass jeder Code darauf zugreifen kann.

Die Variable ist als static markiert, so dass keine Instanz der Klasse benötigt wird.

public static final double PI = 3.141592653589793;

Die Variable ist als final markiert, da sich PI nicht ändert.

Namenskonzvention: Konstante Variablen werden vollständig in Großbuchstaben geschrieben!

```
1 import java.util.Arrays;
2
3 public class StaticInitializerDemo {
4
5     public static void main(String[] args) {
6         System.out.println("Constant with single value:");
7         System.out.println(ConstantWithoutStatInit.CONSTANT_ONE);
8         System.out.println("Constant with array of values:");
9         System.out.println(Arrays.toString(ConstantWithStatInit.CONSTANT_TWO));
10    }
11 }
12
```

STDIN

Input for the program (Optional)

Output:

Constant with single value:

1

Constant with array of values:

Initialization of index 0 with random value 749814247.

Initialization of index 1 with random value 713004593.

[749814247, 713004593]

! Quiz

🔧 Konstanten

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Zur Erstellung einer Konstanten in Java
kennzeichnen Sie eine Variable als statisch
(`static`) und `final`.

Richtig!

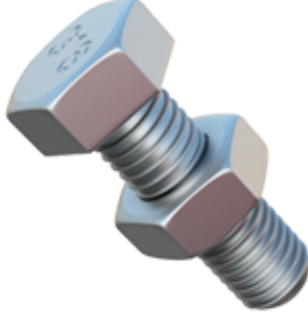
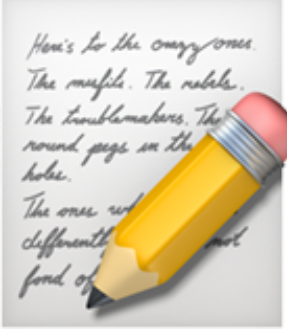
Einer finalen statischen Variablen muss entweder bei ihrer Deklaration oder in einem statischen Initializer ein Wert zugewiesen werden:

```
static {  
    OMNISCIENT_CODE = 42;  
}
```

Richtig!

Die Namenskonvention für Konstanten (finale statische Variablen) ist, dass sie vollständig in Großbuchstaben geschrieben werden und Wörter durch Unterstriche (_) voneinander getrennt werden.

Richtig!

 Welcher **eine** Gedanke zu
„ Konstanten“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren



Schlüsselwort final



Nicht-statische Variablen, Methoden und Klassen

Schlüsselwort **final**

nicht statische finale Variablen

```
class Foof {  
    final int size = 3;  
    final int whuffie;  
  
    Foof() {  
        whuffie = 42;  
    }  
  
    void doStuff(final int x) {  
        // x ist unveränderlich  
    }  
  
    void doMore() {  
        final int z = 7;  
        // z ist unveränderlich  
    }  
}
```

← size kann nicht mehr verändert werden.

← whuffie kann nicht mehr verändert werden.

← // x ist unveränderlich

← // z ist unveränderlich

Wenn eine Variable **final** ist, kann ihr Wert nicht mehr geändert werden.

finale Methode

```
class Poof {  
    final void calcWhuffie() {  
        // wichtige Dinge, die niemals  
        // überschrieben werden dürfen  
    }  
}
```

Wenn eine Methode **final** ist, kann sie nicht überschrieben werden.

finale Klasse

```
final class MyMostPerfectClass {  
    // kann nicht erweitert werden  
}
```

Wenn eine Klasse **final** ist, kann sie nicht erweitert werden. Es können also keine Unterklassen erzeugt werden.

! Quiz

🔑 Schlüsselwort **final**

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Wenn eine finale Variable einmal zugewiesen ist, kann sie nicht mehr geändert werden.

Richtig!

Einer finalen Instanzvariablen muss
entweder bei ihrer Deklaration oder im
Konstruktor ein Wert zugewiesen werden.

Richtig!

Es ist nicht möglich, eine finale Methode zu überschreiben.

Richtig!

Eine finale Klasse kann nicht instanziiert werden (es können keine Objekte erzeugt werden).

... kann nicht erweitert werden (keine Unterklassen).

 Welcher **eine** Gedanke zu
„ Schlüsselwort final“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 **static und final**

🤝 **Pair Programming**

- 🎯 Erstellen Sie eine nicht instanziierbare Klasse, die sowohl eine öffentliche Konstante als auch eine Hilfsmethode bereitstellt.
- ✅ Erstellen Sie eine Klasse `Multiplier`, die eine öffentliche Konstante `FACTOR` enthält.
- ✅ Fügen Sie Ihrer Klasse eine Hilfsmethode `multiply(...)` hinzu, die eine gegebene Zahl mit dem konstanten Faktor multipliziert.



static und final

Pair Programming

- 🎯 **Erstellen Sie eine nicht instanziierbare Klasse, die sowohl eine öffentliche Konstante als auch eine Hilfsmethode bereitstellt.**
- ✅ Erstellen Sie eine Klasse `Multiplier`, die eine öffentliche Konstante `FACTOR` enthält.
- ✅ Fügen Sie Ihrer Klasse eine Hilfsmethode `multiply(...)` hinzu, die eine gegebene Zahl mit dem konstanten Faktor multipliziert.

- ✅ Stellen Sie sicher, dass Ihre Klasse nicht instanziiert werden kann.
- ✅ Wählen Sie geeignete Modifizierer für Klassen, Variablen und Methoden.
- ✅ Schreiben Sie eine Testklasse `MultiplierTestDrive`, die Ihre Klasse und die Hilfsmethode testet.
- 🏁 Am Ende sollten Sie sichergestellt haben, dass das Ergebnis der Multiplikation einem erwarteten Wert entspricht.

```
1 public class MultiplierTestDrive {  
2  
3     public static void main(String[] args) {  
4  
5         final int testNumber = 3;  
6         final int expected = 126; // 3 * 42 = 126  
7         final int actual = Multiplier.multiply(testNumber);  
8  
9         if (expected == actual) {  
10             System.out.println("Test passed: " + testNumber + " * " + Multiplier.FACTOR + " = " + actual);  
11         } else {  
12             System.out.println("Test failed: Expected " + expected + " but got " + actual);  
13         }  
14     }  
15 }  
16
```

STDIN

Input for the program (
Optional)

Output:

Test passed: 3 * 42 = 126

 Welcher **eine** Gedanke zu
„ static und final“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Zahlen und Statisches

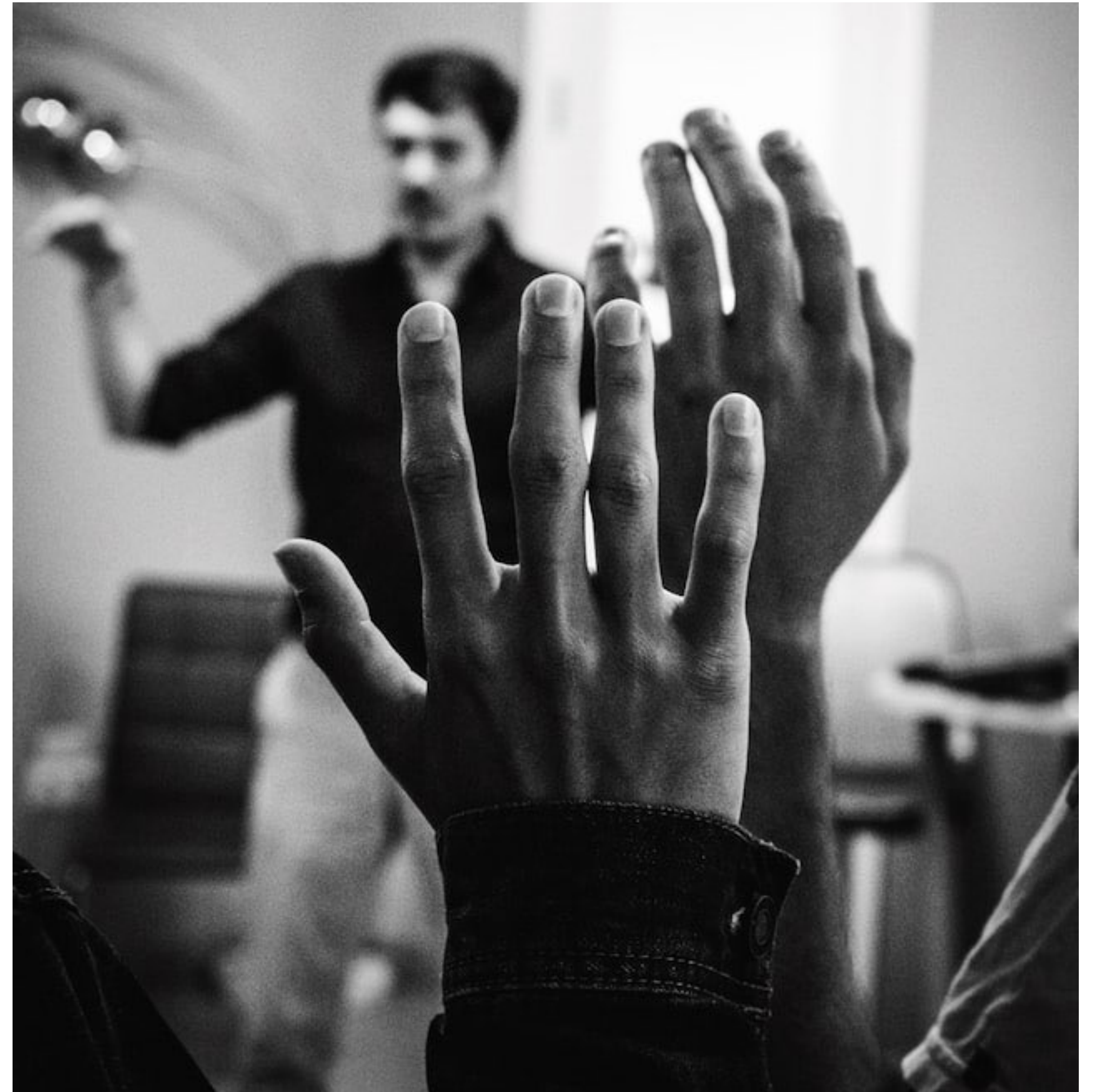
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Zahlen und Statisches?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Zahlen und Statisches



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

